

Final Project Proposal: Racing Line Steering Prediction

Sam Kutsyn, 2581500

Volodymyr Kuchera, 2523181

EE 446

July 22, 2026

Problem Statement

Autonomous racing requires ultra-low-latency perception-to-control pipelines. While full-scale autonomous vehicles rely on heavy compute platforms, small-scale RC racing (e.g., 1/28 scale Kyosho Mini-Z) demands a lightweight, edge-deployable solution due to strict size, weight, and power constraints. Traditional robotics approaches use complex SLAM algorithms to process LiDAR data, which are too computationally expensive for microcontrollers.

This project serves as the foundational software pipeline for a future physical build: a 1/28 scale Kyosho Mini-Z RC car retrofitted for autonomous racing. We evaluated cameras versus distance sensors during our research; while cameras offer rich visual data, they require computationally heavy CNNs and are difficult to deploy on microcontrollers with strict memory limits. Conversely, simple time-of-flight (ToF) sensors lack the forward visibility needed to calculate a racing line. We selected a 2D LiDAR (such as the SLAMTEC RPLIDAR C1) as the optimal tradeoff: it provides a 360-degree geometric understanding of track walls while allowing the use of highly compressible 1D neural networks suitable for TinyML.

This quarter, we aim to build an end-to-end deep neural network (DNN) that maps a 2D LiDAR scan directly to a continuous steering angle. By leveraging TinyML compression techniques, we will shrink this neural network to fit entirely on an Arduino Nano 33 BLE Sense, proving the algorithmic feasibility before tackling the physical hardware constraints.

Datasets

Since physical RC car hardware and LiDAR sensors are not available during this quarter, we will generate a realistic simulated dataset by writing our own simulator.

Simulation Logic

We will define a 2D racetrack using inner and outer wall polygons. We will calculate the optimal “racing line” and then place a virtual car along this line and cast N virtual rays representing a forward-facing 180° LiDAR sweep, mimicking hardware like the SLAMTEC RPLIDAR C1.

Features (X): N continuous distance values representing the distance from the car to the track walls.

Labels (Y): The continuous steering angle (-1.0 to 1.0) required to stay on the racing line at that specific position.

Visualization

We also hope to add a visual component by rendering a top-down visualizer. The simulator will plot the track walls, the virtual car’s position, the N cast LiDAR rays, and a visual arrow indicating the ground-truth steering angle. This ensures our dataset is highly interpretable and provides a visual interface for our live demo.

Modeling Approach

Following the Choosing DNN Architectures for TinyML Tasks guidelines, this project is a Nonlinear Regression task. The output layer will consist of a single node with a Linear activation, and the model will be trained using MSE loss.

To satisfy TinyML constraints and explore the Compression Cookbook, we will employ a Teacher-Student paradigm:

- Teacher Model: A 1D Convolutional Neural Network (CNN) capable of extracting complex spatial patterns from the N-value LiDAR sweep.
- Student Model: A much smaller Feedforward Neural Network (FFNN) (e.g., $60 \rightarrow 32 \rightarrow 16 \rightarrow 1$) that is highly deployable on a microcontroller.

Compression Strategy

Knowledge Distillation We will train the large 1D CNN Teacher, then distill its knowledge into the tiny FFNN Student using a custom Keras Distiller wrapper to retain high accuracy despite a drastically reduced parameter count.

Pruning We will apply polynomial decay magnitude-based pruning to the distilled student model, targeting roughly 80-85% sparsity. We will strip the pruning mask and enable experimental sparsity optimization during TFLite conversion.

Quantization We will apply Post-Training Quantization (PTQ) to Int8. If regression accuracy degrades too much due to integer discretization, we will apply Quantization-Aware Training (QAT) to recover steering precision before final conversion.

Deployment Plan

Our target hardware is the Arduino Nano 33 BLE Sense. Because the Arduino Nano does not have a physical LiDAR sensor attached, our laptop will act as the “car environment” for the live demo.

Our Python visualizer will stream the simulated N-value LiDAR arrays to the Arduino over a serial USB connection. The Arduino will receive the array, run inference using the final model, and send the predicted steering angle back to the laptop. The laptop will update the visualizer in real-time, animating the virtual car driving around the track entirely controlled by the TinyML model on the microcontroller.

Challenges & Solutions

Int8 Quantization Sensitivity for Regression

Continuous regression outputs are highly sensitive to the discretization introduced by Int8 quantization, which could result in jerky or biased steering predictions.

Solution: We will use a highly representative dataset during PTQ calibration. If accuracy drops, we will implement Quantization-Aware Training (QAT) to fine-tune the model with fake quantization nodes, allowing the network to adapt to integer constraints before deployment.

Sim-to-Reality Gap and Sensor Noise

Simulated LiDAR rays are perfectly clean, whereas physical sensors introduce noise, missed readings, and distance outliers.

Solution: To prepare the model for future deployment on a physical 1/28 scale RC car, we will inject Gaussian noise and random dropouts into the simulated distance arrays during the training phase. This data augmentation will force the neural network to learn robust racing line prediction rather than memorizing perfect distances.